

```

// =====
// MuchaBot - מוצ'ה ישראל - שרת טלגרם למסעדת מוצ'ה ישראל
// Node.js + Express + Telegram Bot API + Claude AI
// =====

const express = require("express");
const axios = require("axios");
const app = express();
app.use(express.json());

// --- הגדרות ---
const TELEGRAM_TOKEN = process.env.TELEGRAM_TOKEN; // Bot Token n-BotFather
const ANTHROPIC_KEY = process.env.ANTHROPIC_KEY; // API Key n-Anthropic
const MANAGER_CHAT_ID = process.env.MANAGER_CHAT_ID; // Chat ID (המנהל)
const BASE_URL = `https://api.telegram.org/bot${TELEGRAM_TOKEN}`;

const DAYS = ["ראשון", "שני", "שלישי", "רביעי", "חמישי", "שישי", "שבת"];

// --- חיצוני DB פשוט, ללא מסד נתונים בזיכרון ---
// employees: { [chatId]: { name, role, availability: [], constraints: "", done:
// בעתיד Redis/MongoDB - כדי שהנתונים לא יאבדו בהפעלה מחדש - אפשר להחליף ב
let employees = {};
let scheduleState = "idle"; // idle | collecting | done

// --- שליחת הודעה ---
async function sendMessage(chatId, text, keyboard = null) {
  const body = { chat_id: chatId, text, parse_mode: "HTML" };
  if (keyboard) body.reply_markup = keyboard;
  await axios.post(`${BASE_URL}/sendMessage`, body);
}

// --- כפתורי ימים (Inline Keyboard) ---
function daysKeyboard(selected = []) {
  const rows = [];
  for (let i = 0; i < DAYS.length; i += 3) {
    rows.push(
      DAYS.slice(i, i + 3).map(day => ({
        text: selected.includes(day) ? `✅ ${day}` : day,
        callback_data: `day_${day}`,
      }))
    );
  }
  rows.push([
    { text: "✅ סיימתי!", callback_data: "done" }
  ]);
  return { inline_keyboard: rows };
}

```

```
}
```

```
// — Claude בניית סידור עם —————
```

```
async function buildSchedule() {
  const empList = Object.values(employees);
  const availabilityText = empList
    .map(e => `${e.name} (${e.role}): זמין-${e.availability.join(", ")}${e.cons
    .join("\n");
```

```
  const prompt = `מנהל מסעדה. בנה סידור עבודה שבועי אופטימלי למסעדת "מוצ'ה ישראל`
```

```
נתוני זמינות:
```

```
${availabilityText}
```

```
כללים:
```

- כל משמרת: לפחות שף/עוזר שף אחד ומלצר/ית אחד
- פזר עובדים בצורה הוגנת
- כבד אילוצים אישיים

```
החזר JSON (לא backticks):
```

```
{
  "schedule": {
    ["שם1", "שם2"] : "ראשון",
    ["שם3"] : "שני",
    "שלישי": [],
    ["שם1"] : "רביעי",
    ["שם2", "שם3"] : "חמישי",
    ["שם1", "שם2"] : "שישי",
    ["שם1", "שם4"] : "שבת"
  },
  "notes": "הערה קצרה"
};`;
```

```
const res = await axios.post(
  "https://api.anthropic.com/v1/messages",
  {
    model: "claude-sonnet-4-20250514",
    max_tokens: 1000,
    messages: [{ role: "user", content: prompt }],
  },
  {
    headers: {
      "x-api-key": ANTHROPIC_KEY,
      "anthropic-version": "2023-06-01",
      "content-type": "application/json",
    },
  }
);
```

```

);

const text = res.data.content?.[0]?.text || "{}";
return JSON.parse(text.replace(/``json|``/g, "").trim()));
}

// — פורמט סידור לטקסט —
function formatSchedule(schedule, notes) {
  let msg = "📅 <b>סידור עבודה שבועי - מוצ'ה ישראל</b>\n\n";
  for (const day of DAYS) {
    const workers = schedule[day] || [];
    msg += `<b>${day}</b>: ${workers.length ? workers.join(", ") : "-"}\n`;
  }
  if (notes) msg += `\n💡 ${notes}`;
  return msg;
}

// — שלח סידור אישי לכל עובד —
async function sendPersonalSchedule(chatId, name, schedule) {
  const myDays = Object.entries(schedule)
    .filter(([, names]) => names.includes(name))
    .map(([day]) => day);

  const msg = myDays.length > 0
    ? `📅 <b>הסידור שלך לשבוע הבא</b>\n\n${myDays.map(d => `• ${d}`).join("\n")}`
    : `השבוע אין לך משמרות מתוכננות. נדבר, שלום! 📞`;

  await sendMessage(chatId, msg);
}

// — webhook —
app.post("/webhook", async (req, res) => {
  res.sendStatus(200); // תמיד תגיב מהר לטלגרם

  const body = req.body;

  // — הודעת טקסט —
  if (body.message) {
    const msg = body.message;
    const chatId = String(msg.chat.id);
    const text = msg.text || "";
    const firstName = msg.from.first_name || "חבר";

    // /start - הרשמה
    if (text === "/start") {
      if (!employees[chatId]) {
        employees[chatId] = {

```

```

        name: firstName,
        role: "עובד",
        availability: [],
        constraints: "",
        done: false,
    };
}
await sendMessage(chatId,
    `מוצי'ה ישראל<b> </b> ברוך הבא לבוט של מסעדתה! 🙌\n\n
    שלום ${firstName}!`;
// עדכן מנהל
if (MANAGER_CHAT_ID) {
    await sendMessage(MANAGER_CHAT_ID, `✅ עובד חדש הצטרף: <b>${firstName}</b>
}
return;
}

// /collect - זמין איסוף זמין
if (text === "/collect" && chatId === MANAGER_CHAT_ID) {
    scheduleState = "collecting";
    // איפוס זמין
    Object.keys(employees).forEach(id => {
        employees[id].availability = [];
        employees[id].constraints = "";
        employees[id].done = false;
    });

    const empIds = Object.keys(employees);
    if (empIds.length === 0) {
        await sendMessage(chatId, "❌ /sta
        return;
    }

    for (const id of empIds) {
        const emp = employees[id];
        await sendMessage(id,
            `שלום ${emp.name}! 📅\n אתה מבקשת אתה ישראל מוצ'ה מסעדת מוצ'ה ישראל
            daysKeyboard([])
        );
    }
    await sendMessage(chatId, `📅-שלום זמין ל-$ {empIds.length}
    return;
}

// /status - מצב רואה
if (text === "/status" && chatId === MANAGER_CHAT_ID) {
    const total = Object.keys(employees).length;

```

```

const done = Object.values(employees).filter(e => e.done).length;
let msg = `📊 <b>חצב איסוף זמינות:</b>\n\n`;
msg += `✅ ענו: ${done}/${total}\n\n`;
Object.values(employees).forEach(e => {
  msg += `${e.done ? "✅" : "⌚"} ${e.name}: ${e.availability.length ? e.av
});
await sendMessage(chatId, msg);
return;
}

// /build – מנהל מבקש לבנות סידור ידנית
if (text === "/build" && chatId === MANAGER_CHAT_ID) {
  await sendMessage(chatId, "🤖 AI הסידור את הבונה...");
  try {
    const result = await buildSchedule();
    const fullSchedule = formatSchedule(result.schedule, result.notes);
    await sendMessage(chatId, fullSchedule);

    // שלח סידור אישי לכל עובד
    for (const [id, emp] of Object.entries(employees)) {
      await sendPersonalSchedule(id, emp.name, result.schedule);
    }
    await sendMessage(chatId, "✅ הסידור נשלח לכל העובדים!");
    scheduleState = "done";
  } catch (err) {
    await sendMessage(chatId, `❌ שגיאה: ${err.message}`);
  }
  return;
}

// /workers – רשימת עובדים
if (text === "/workers" && chatId === MANAGER_CHAT_ID) {
  const list = Object.entries(employees)
    .map(([id, e]) => `• ${e.name} (${e.role}) – ID: ${id}`)
    .join("\n");
  await sendMessage(chatId, `👥 <b>עובדים רשומים:</b>\n\n${list} || "עובדים"
  return;
}

// הודעת טקסט חופשית – אילוף אחרי בחירת ימים
if (employees[chatId] && employees[chatId].done === false && employees[chatId]
  employees[chatId].constraints = text;
  await sendMessage(chatId, `✅ זמינות! רשמתי את האילוף שלך: ${employees[cha
}
}

// — לחיצה על כפתור —

```

```

if (body.callback_query) {
  const cb = body.callback_query;
  const chatId = String(cb.message.chat.id);
  const data = cb.data;
  const messageId = cb.message.message_id;

  // מעצבן popup אישור ללא
  await axios.post(`${BASE_URL}/answerCallbackQuery`, { callback_query_id: cb.i

  if (!employees[chatId]) return;
  const emp = employees[chatId];

  if (data.startsWith("day_")) {
    const day = data.replace("day_", "");
    if (emp.availability.includes(day)) {
      emp.availability = emp.availability.filter(d => d !== day);
    } else {
      emp.availability.push(day);
      emp.availability.sort((a, b) => DAYS.indexOf(a) - DAYS.indexOf(b));
    }
  }

  // עדכן את הכפתורים
  await axios.post(`${BASE_URL}/editMessageReplyMarkup`, {
    chat_id: chatId,
    message_id: messageId,
    reply_markup: daysKeyboard(emp.availability),
  });
}

if (data === "done") {
  if (emp.availability.length === 0) {
    await sendMessage(chatId, "⚠️אנא בחר לפחות יום אחד לפני שתסיים");
    return;
  }

  emp.done = true;
  await sendMessage(chatId,
    `תודה ${emp.name}! ✅\n\nלרשומתי אותך: <b>${emp.availability.join(", ")}<
  );

  // עדכן מנהל
  if (MANAGER_CHAT_ID) {
    const done = Object.values(employees).filter(e => e.done).length;
    const total = Object.keys(employees).length;
    await sendMessage(MANAGER_CHAT_ID,
      `📊 ${emp.name} ענה! (${done}/${total})\nזמינות: ${emp.availability.joi
    );
  }

```

```

// כולם ענו - בנה סידור אוטומטי
if (done === total) {
  await sendMessage(MANAGER_CHAT_ID, "🎉 סידור אוטומטי! מחר יל לבנות סידור אוטומטי");
  setTimeout(async () => {
    try {
      const result = await buildSchedule();
      const fullSchedule = formatSchedule(result.schedule, result.notes);
      await sendMessage(MANAGER_CHAT_ID, fullSchedule);
      for (const [id, e] of Object.entries(employees)) {
        await sendPersonalSchedule(id, e.name, result.schedule);
      }
      await sendMessage(MANAGER_CHAT_ID, "✅ הסידור נשלח לכל העובדים");
      scheduleState = "done";
    } catch (err) {
      await sendMessage(MANAGER_CHAT_ID, "❌ שגיאה בבניית הסידור: ${err.m
    }
  }, 2000);
}
}
}
});

// — Health check —————
app.get("/", (req, res) => res.send("MuchaBot! 🍕"));

// — הפעלה —————
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`✅ MuchaBot פורט על ${PORT}`));

```